Button Detection

1. Button Detection Overview

Buttons are the most common human-machine interaction input devices. The MCU determines the button state by reading the GPIO input level. This experiment uses the BOOT button (GPIO0) to control LED on/off toggling, demonstrating GPIO input configuration, software debouncing, and falling edge detection. Each press toggles once, and holding does not repeat triggering.

Typical Application Scenarios:

Application Type	Example
Digital Input	User buttons, switch state reading, sensor digital signals, rotary encoders
Pull-Up Input	Button connected to GND, limit switches, reed switches, photoelectric switches
Debounce Processing	Mechanical buttons, relay contact detection, DIP switches

2. Core Concepts

2.1 GPIO Input Mode

When GPIO is configured in input mode, the pin presents **high impedance state**, and the level is determined by the external circuit.

Key Functions of Pull-Up/Pull-Down Resistors:

Configuration	GPIO Default Level	When Button Pressed	Applicable Scenario
Internal Pull-Up (Pull-Up)	High (1)	Low (0)	Button one end connected to GND, conducts when pressed
Internal Pull-Down (Pull-Down)	Low (0)	High (1)	Button one end connected to VDD, conducts when pressed
No Pull-Up/Pull-Down	Uncertain (floating)	Uncertain	**Not recommended**, floating pins are susceptible to interference

This experiment uses **internal pull-up** (`INPUT_PULLUP`), with one end of the button connected to GND and the other end to GPIO0:

```
VDD ──→ Internal pull-up resistor (~45kΩ) ──→ GPIO0 ──→ Button ──→ GND

Button released: GPIO0 pulled up to VDD → Read 1 (HIGH)
Button pressed: GPIO0 directly connected to GND → Read 0 (LOW)
```

2.2 Mechanical Button Bounce

When mechanical buttons are pressed and released, metal contacts experience **bouncing**, producing a string of irregular high/low level pulses:

```
Ideal waveform:  ─────┐             ┌─────
                      └─────────────┘

Actual waveform:  ─────┐ ┌ ┌ ┌ ┌ ┌ ┌ ┌ ┌─────
                       └ └ └ └ └ └ └ └
                        ├─ Bounce period ─┤
                       (typical 5~20ms)
```

Without debouncing, the MCU may misinterpret one press as multiple triggers. Arduino uses **falling edge detection + delay debouncing**: after detecting the button changing from high to low (falling edge), it uses `delay(20)` to skip the entire bounce period, which is simple and efficient.

2.3 Falling Edge Detection Principle

```
loop() each iteration:
  keyState = digitalRead(KEY_PIN)     // Read current level
  |
  +-- keyState == LOW  &&  lastKeyState == HIGH ?
      |
      YES → Falling edge confirmed → Toggle LED → delay(20) debounce
      NO  → Holding or bouncing, no action
      |
  lastKeyState = keyState            // Save this reading for next comparison
```

`keyState`	`lastKeyState`	**Result**
LOW (0)	HIGH (1)	**Falling edge — Trigger toggle**
LOW (0)	LOW (0)	Holding — No trigger
HIGH (1)	HIGH (1)	Releasing — No trigger
HIGH (1)	LOW (0)	Rising edge — No trigger

> **Why can `delay(20)` debounce?** After confirming the falling edge, `delay(20)` makes the program "pause" for 20ms, which exactly covers the 5~20ms bounce period. When `delay` returns, bouncing has ended, and `loop()` resumes normal polling. The trade-off is that the CPU is completely blocked during `delay`, which is perfectly adequate for simple projects.

---

3. Hardware Dependencies

Pin	Peripheral	Configuration	Description
GPIO43	LED	`OUTPUT`	Common anode connection, low level lights up
GPIO0	BOOT button	`INPUT_PULLUP`	Pressed = low level, released = high level

> GPIO0 is also a Strapping pin, which affects the boot mode during power-on. **After normal operation**, using it as button input is fine, but external circuits should not force GPIO0 low, otherwise the chip may fail to boot.

---

4. Project Architecture and Flow

```
setup()                          // Executes only once
  ├─ pinMode(LED_PIN, OUTPUT)    // LED push-pull output
  └─ pinMode(KEY_PIN, INPUT_PULLUP) // Button pull-up input

loop()                           // Infinite loop
  ├─ keyState = digitalRead()    // Read current level
  ├─ if (keyState==LOW && lastKeyState==HIGH)  // Falling edge?
  |    ├─ ledState = !ledState   // Toggle LED state
  |    ├─ digitalWrite(ledState)
  |    └─ delay(20)              // Skip bounce period
  └─ lastKeyState = keyState     // Save state → Back to start
```

Key Design Principles:

- `ledState` saves the LED output level (`LOW`=on, `HIGH`=off), and `!ledState` toggles to switch
- `lastKeyState` records the previous button level, which is the foundation of edge detection — forgetting to update it will cause "falling edge" to be detected every cycle when holding
- The main loop has no blocking `delay` (except for the 20ms debounce), ensuring timely button response

---

5. Source Code Explanation

Full source code: `Source Code → Arduino → 1.Basic Peripherals → 2.KEY_LED → 2.KEY_LED.ino`

5.1 Macro Definitions and Global Variables

First define the pin numbers, then use two global variables to remember the LED current state and the previous button level.

```
#define LED_PIN     43  // Onboard LED
#define KEY_PIN     0   // BOOT button, pressed to ground

int ledState = LOW;           // LED current output level, LOW=on (common anode)
int lastKeyState = HIGH;      // Previous button level, initial HIGH=not pressed
```

> **Why is the initial value of `lastKeyState` `HIGH`?** Because `INPUT_PULLUP` is used, when the button is not pressed, the pin is pulled up to high level. Initializing to `HIGH` is consistent with the physical state, avoiding false falling edge detection in the first `loop()` cycle after power-on.

5.2 `setup()` — GPIO Initialization

LED is set to push-pull output, button is set to pull-up input. `INPUT_PULLUP` uses one macro to complete both "input mode" and "enable pull-up" configurations.

```
void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(KEY_PIN, INPUT_PULLUP);
}
```

> When using `INPUT` (no pull-up/pull-down), the pin is floating, and `digitalRead()` reads random noise. This is the most common pitfall for beginners — remember button detection must include `_PULLUP`.

5.3 `loop()` — Main Loop and Button Detection (Core)

First read the button, then check for falling edge — if current is low and previous is high, the button has just been pressed. After confirmation, toggle the LED and delay 20ms for debouncing, finally save this reading for the next cycle comparison.

```
void loop() {
  int keyState = digitalRead(KEY_PIN);

  // Falling edge: currently pressed(0) and previously released(1)
  if (keyState == LOW && lastKeyState == HIGH) {
    ledState = !ledState;             // Toggle
    digitalWrite(LED_PIN, ledState);
    delay(20);                        // Skip bounce period
  }

  lastKeyState = keyState;            // Save for next comparison
}
```

Execution flow analysis (for a complete button press):

```
Timeline         GPIO0 Level       loop() Behavior
──────────────────────────────────────────────────
0ms              1 (released)      Read=1, condition not met → lastKeyState=1
...continuous polling...
100ms            0 (just pressed)    Read=0, lastKeyState=1 → Falling edge!
                                  Toggle LED → delay(20) → lastKeyState=0
100~300ms        0 (held)             Read=0, lastKeyState=0 → Condition not met
300ms            1 (released)      Read=1, lastKeyState=0 → Rising edge, no action
```

> **Why must `lastKeyState = keyState` be outside the `if`?** If it were only inside the `if`, when a falling edge is captured (short press, long press, or holding), `lastKeyState` would always be `LOW`, and every cycle would detect "falling edge". After that, whether holding or repeatedly pressing/releasing the button, the edge detection condition would never be met again, and the button function would completely fail.

---

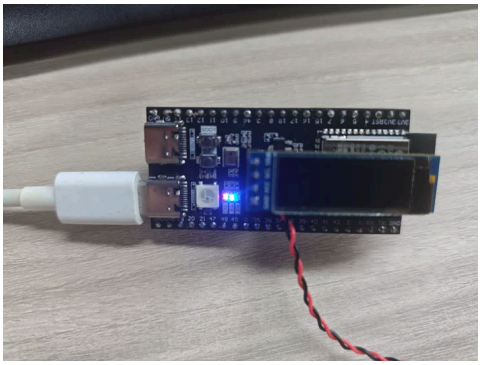
6. Operation Flow

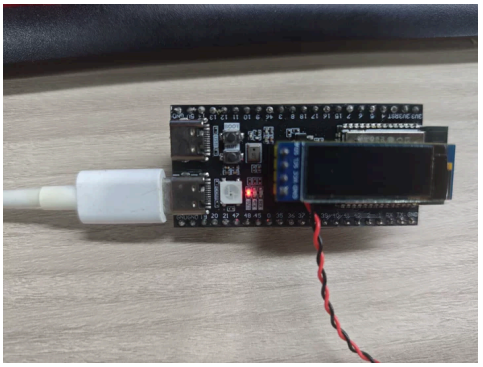
6.1 Build and Flash

Arduino flash process: See `Arduino → 1. Before Development → 2. Build and Flash`.

6.2 Normal Operation

- After power-on, the LED is initially **on** (`ledState` initial value is `LOW`, common anode = on)
- Each press of the BOOT button → LED toggles once (on→off or off→on)
- **Holding without releasing**: Toggles only once, next toggle occurs on next press after release





6.3 Boundary Testing

Test Item	Operation	Expected Result
Short press	Quickly click BOOT button and release immediately	LED toggles once
Long press	Hold BOOT button for 5 seconds	LED toggles once (no repeated toggling while holding)
Rapid consecutive presses	Continuously click 10 times quickly	Each valid press toggles once
Power-on default	Power on without pressing button	LED lights up (`ledState` initial `LOW`)

---

7. Common Issues

Phenomenon	Cause	Solution
One press toggles multiple times	Insufficient debounce time	Increase `delay(20)` to 30~50ms
Button response sluggish	Debounce delay too large	Reduce `delay()` , recommended no less than 10ms
Button completely unresponsive	GPIO0 configuration error / no pull-up	Confirm `pinMode(KEY_PIN, INPUT_PULLUP)`
Toggles on its own without pressing	GPIO0 floating + no pull-up/pull-down → random level	Use `INPUT_PULLUP` , do not use `INPUT`
LED on/off logic reversed	`ledState` initial value not as expected	Change `int ledState = LOW` to `= HIGH`